Data Partitioning

Introduction

- Data replication
  - Not sufficient for very large datasets
  - Solution: Break the data up into *partitions*, also known as *sharding*
- Partition
  - Shard in MongoDB, Elasticsearch, and SolrCloud
  - Region in Hbase
  - Tablet in Bigtable
  - Vnode in Cassandra and Riak
  - vBucket in Couchbase

Introduction

- Each partition is a small database of its own.
- Each piece of data (row or document) belongs to exactly one partition.
- The main reason for wanting to partition data is *scalability*.
  - A large dataset can be distributed across many disks.
  - The query load can be distributed across many processors.

Partitioning and Replication

- Copies of each partition are stored on multiple nodes.
- Each record belongs to one partition but may be stored on multiple nodes for *fault tolerance*.
- A node may store more than one partition.
- If a leader–follower replication model is used, the combination of partitioning and replication can look like Figure 6-1.

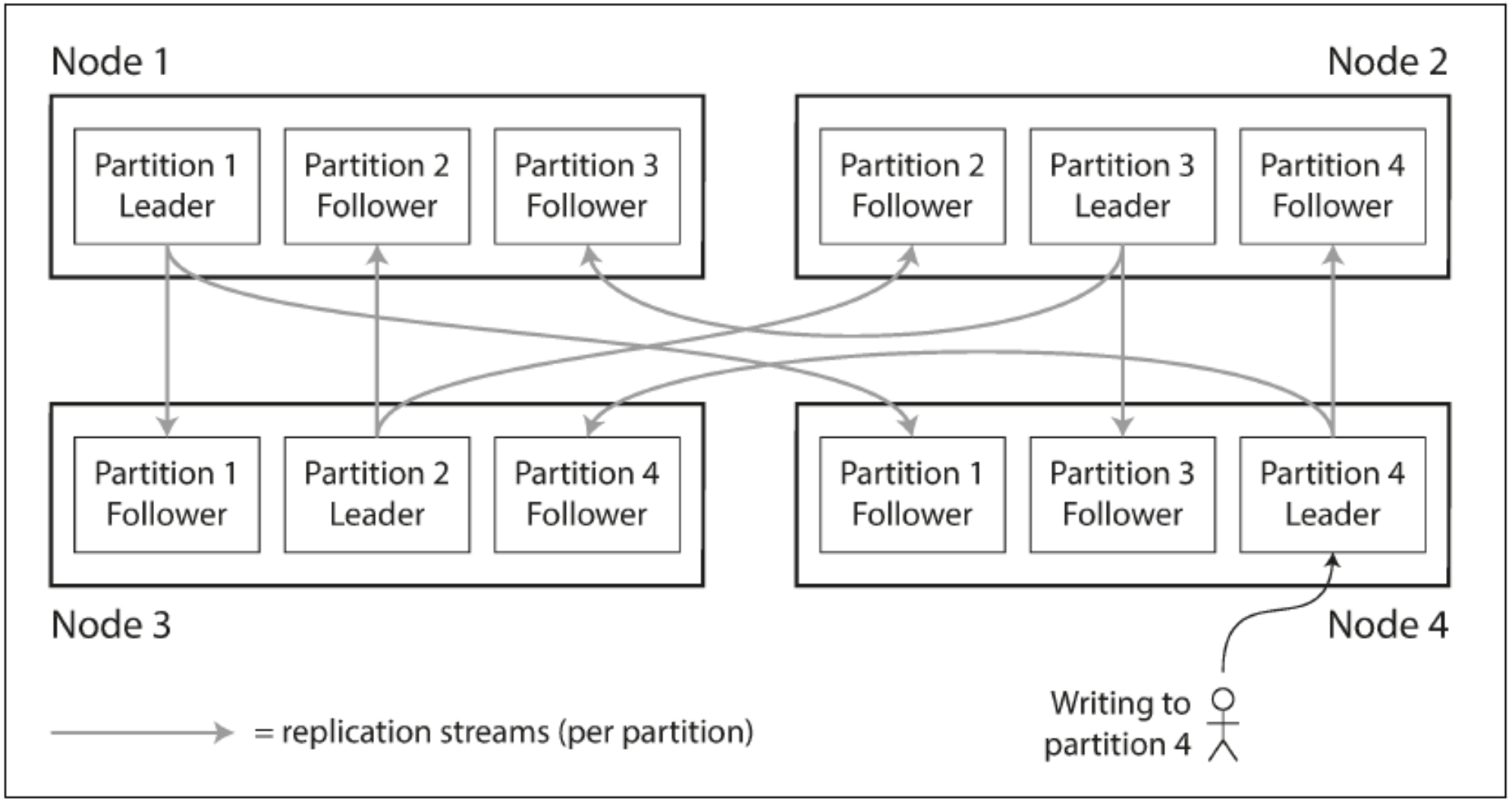


Figure 6-1. Combining replication and partitioning: each node acts as leader for some partitions and follower for other partitions.

Partitioning and Replication

- The choice of partitioning scheme is mostly independent of the choice of replication scheme.

Partitioning of Key-Value Data

- The goal: Spread the data and query load evenly across nodes - fair partitioning
- If every node takes a fair share, then—in theory—10 nodes should be able to handle 10 times as much data and 10 times the read and write throughput of a single node
- Unfair partitioning – skewed
  - Some partitions have more data or queries than others
- A partition with disproportionately high load is called a hot spot.

Partitioning of Key-Value Data:

Partitioning by Key Range

- Assigning continuous ranges of keys to partitions
- A partition owns all the keys from some minimum up to some maximum

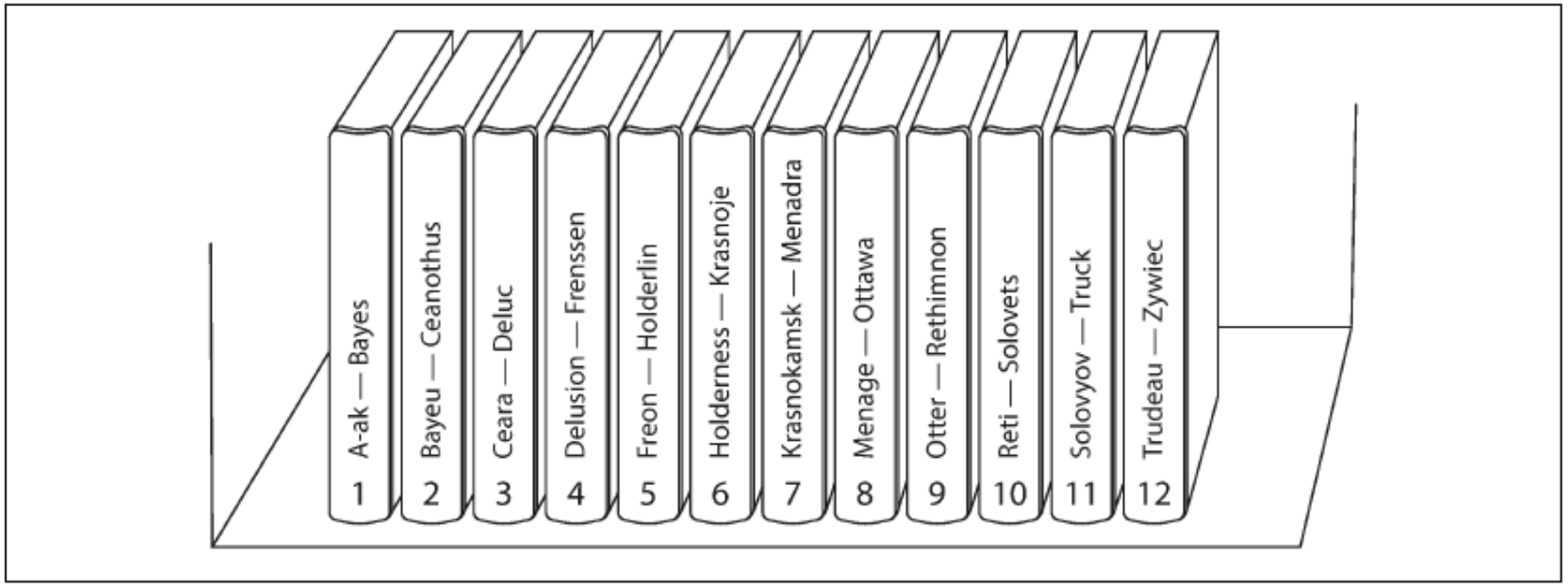


Figure 6-2. A print encyclopedia is partitioned by key range.

Partitioning of Key-Value Data:

Partitioning by Key Range

- If the data is not evenly distributed, the ranges of keys are not evenly spaced.
- To distribute the data evenly, the partition boundaries need to adapt to the data.

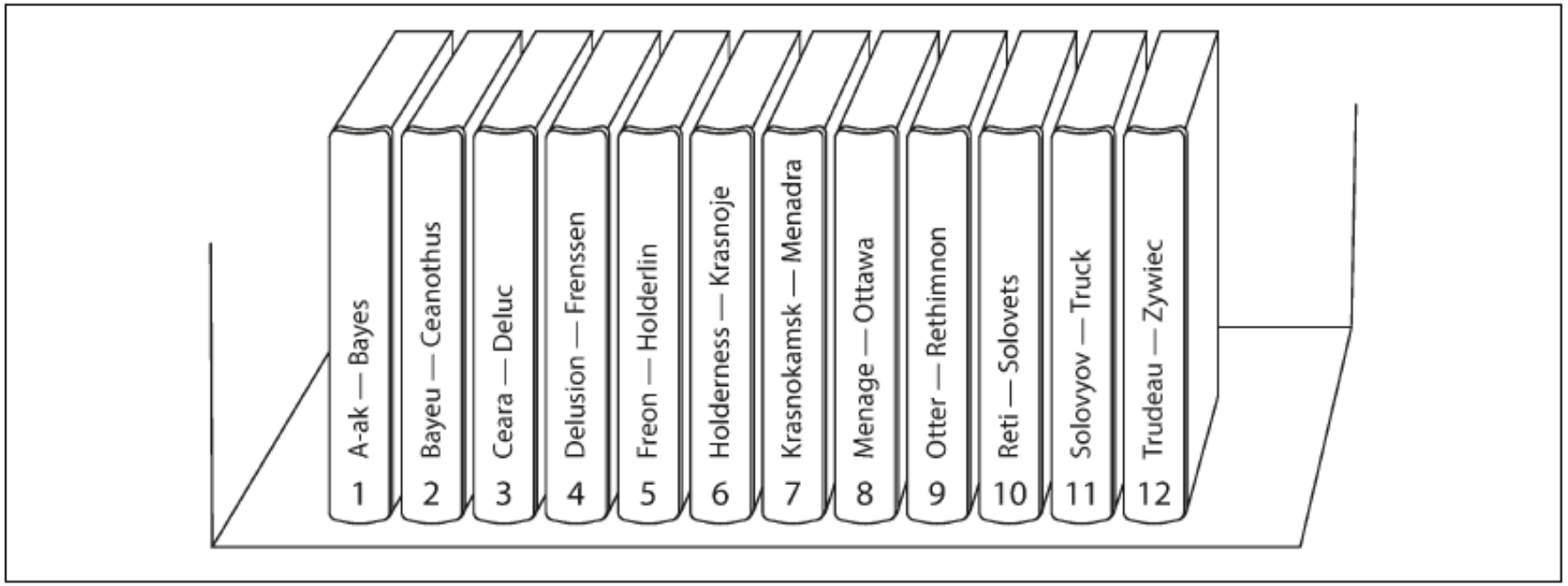


Figure 6-2. A print encyclopedia is partitioned by key range.

Partitioning of Key-Value Data:

Partitioning by Key Range

- Partition boundaries can be chosen manually by an administrator, or the database can choose them automatically.
- Data can be stored in sorted order within each partition.
  - Makes range queries efficient.
- The downside: Certain access patterns can lead to hot spots.

Partitioning of Key-Value Data:

Partitioning by Hash of Key

- Due to the risk of skew and hot spots – many distributed datastores use a hash function.
- Hash function uniformly distributes skewed data.
- A hash function is applied to each key.
- A partition owns a range of hashes.
- Range queries are now difficult (rather than a range of keys).
- This technique is good at distributing keys fairly among the partitions.

Partitioning of Key-Value Data:

Partitioning by Hash of Key

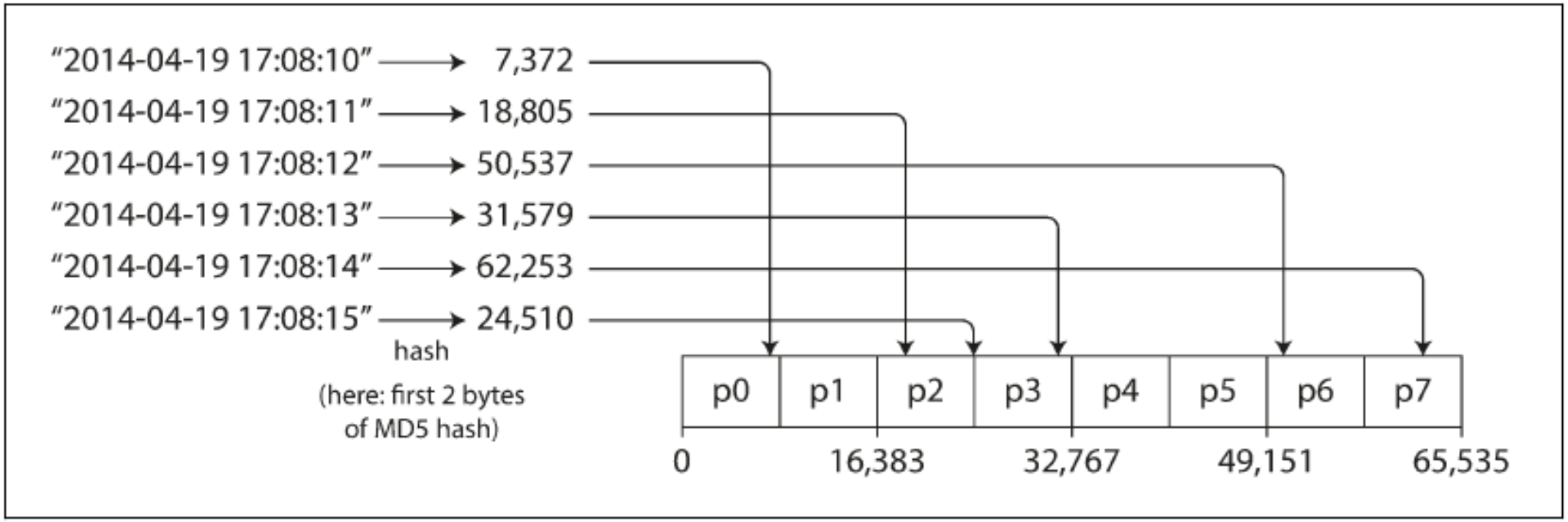


Figure 6-3. Partitioning by hash of key.

Partitioning of Key-Value Data:

Partitioning by Hash of Key

- The ability to do efficient range queries:
  - Key-range partitioning vs hash-key partitioning?
  - Keys are adjacent in key-range partitioning.
  - Keys are scattered across all the partitions in hash-key partitioning.
    - So, their sort order is lost.

Skewed Workloads and Relieving Hot Spots

- Hashing a key to determine its partition can help reduce hot spots.
  - However, it can’t avoid them entirely.
- If you have an extreme number of requests to the same key, you still have a hot spot.
  - Ex: On a social media site, a celebrity user with millions of followers may cause a storm of activity when they do something.
  - Can result in a large volume of writes to the same key.

Skewed Workloads and Relieving Hot Spots

- Most systems cannot automatically solve this type of situation.
  - It's the responsibility of the application to reduce the skew.
  - Ex: If one key is known to be very hot: add a random number to the beginning or end of the key.
  - But reads – have to do additional work.
  - You also need some way of keeping track of which keys are being split.
- You need to think through the trade-offs for your own application.

Partitioning and Secondary Indexes

- A secondary index usually doesn't identify a record uniquely but rather is a way of searching for occurrences of a particular value:
  - "find all actions by user 123"
  - "find all articles containing the word *data*"
  - "find all cars whose color is red"
- The problem with secondary indexes: they don't map neatly to partitions.
- Partitioning secondary indexes by document
- Partitioning secondary indexes by term

Partitioning and Secondary Indexes:

Partitioning Secondary Indexes by Document

- Ex: A website for selling used cars
- Each listing has a unique document ID
- The database is partitioned by the document ID
  - IDs 0 to 499 in partition 0
  - IDs 500 to 999 in partition 1

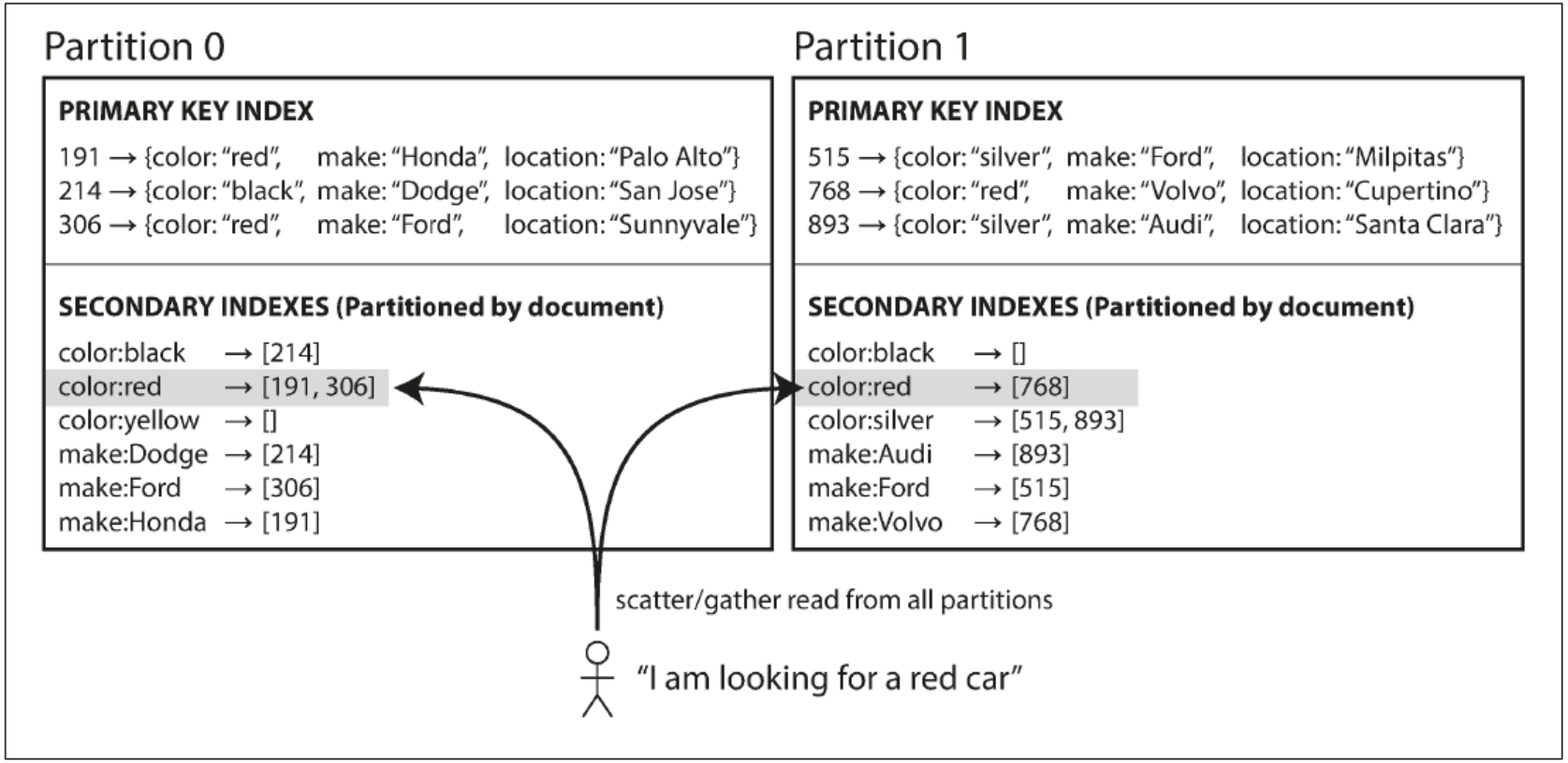


Figure 6-4. Partitioning secondary indexes by document.

Partitioning and Secondary Indexes:

Partitioning Secondary Indexes by Document

- You want to let users search for cars, allowing them to filter by *color* and by *make*
- So, you need a secondary index on *color* and *make*
  - In a document database: fields
  - In a relational database: columns

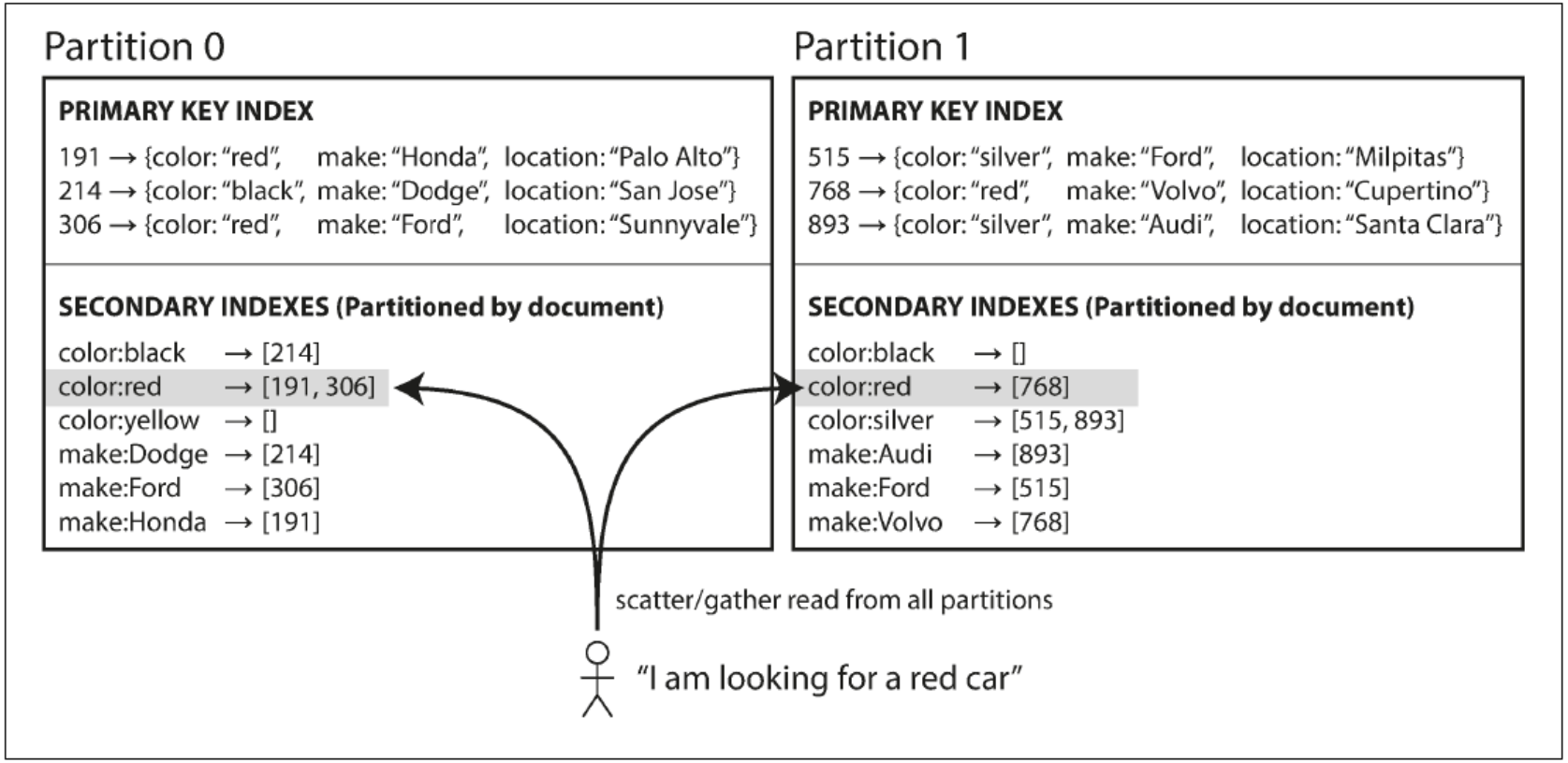


Figure 6-4. Partitioning secondary indexes by document.

Partitioning and Secondary Indexes:

Partitioning Secondary Indexes by Document

- If you have declared the index, the database can perform the indexing automatically.

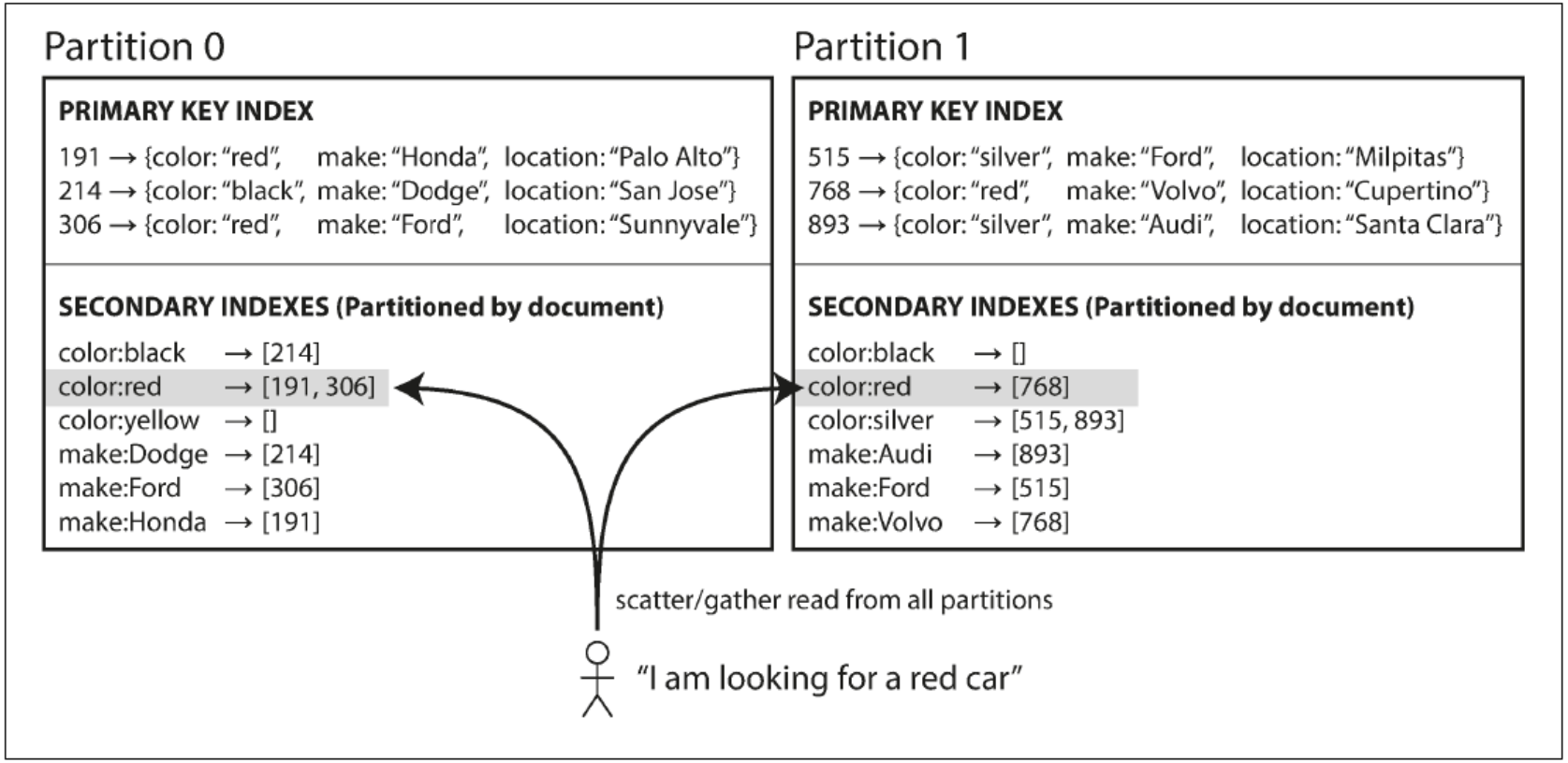


Figure 6-4. Partitioning secondary indexes by document.

Partitioning and Secondary Indexes:

Partitioning Secondary Indexes by Document

- In this indexing approach, each partition is separate
  - Each partition maintains its own secondary indexes for keys on that partition
  - Local index

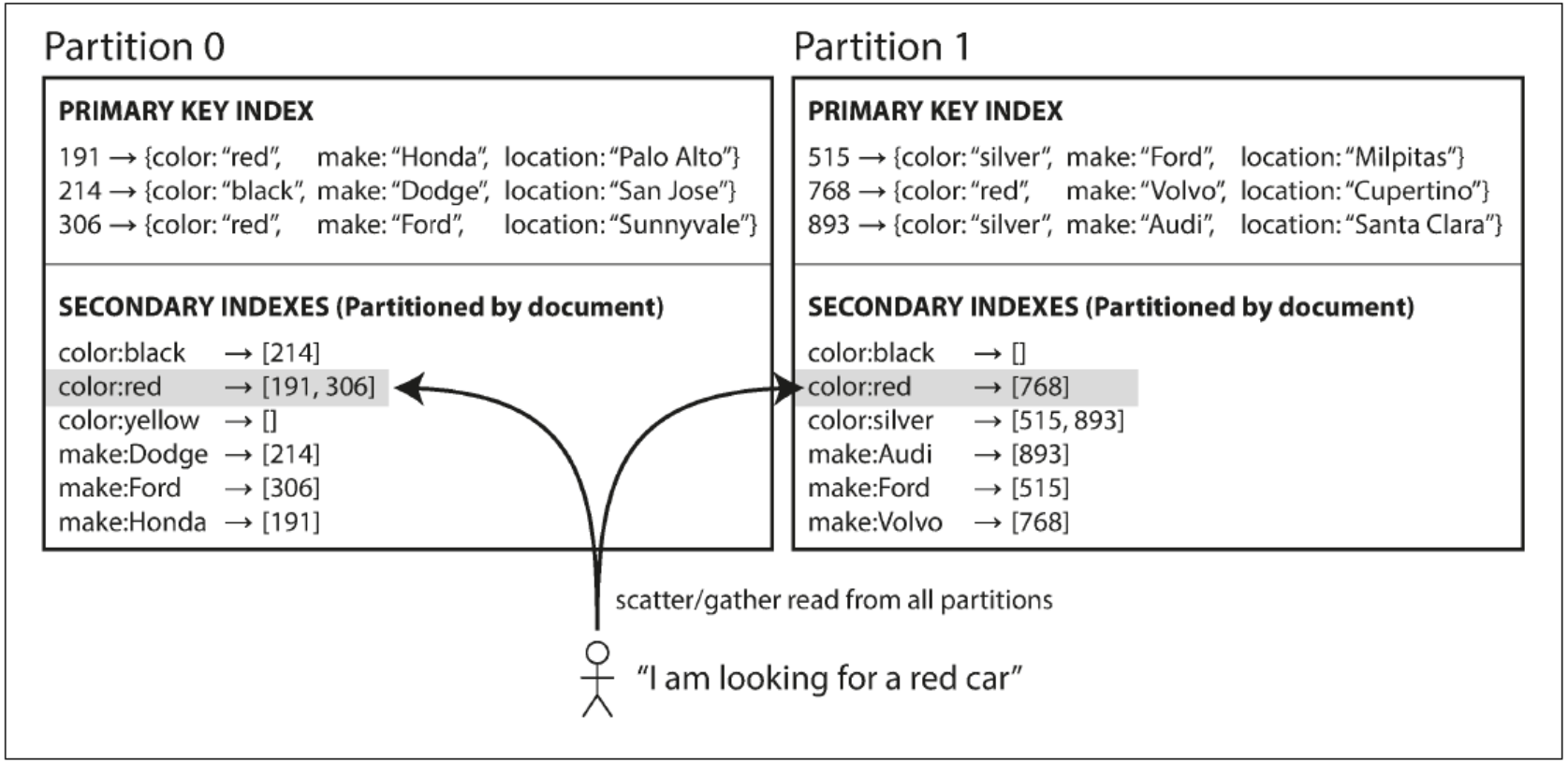


Figure 6-4. Partitioning secondary indexes by document.

Partitioning and Secondary Indexes:

Partitioning Secondary Indexes by Document

- Writes are easy because
  - You only need to deal with the partition that contains the document ID that you are writing.
- Reads must be sent to every partition, called *scatter/gather*
  - Because you must synchronously wait for every partition to respond, scatter/gather is prone to tail latency amplification.
  - Nevertheless, it is widely used: MongoDB, Riak, Cassandra, Elasticsearch,…

Partitioning and Secondary Indexes:

Partitioning Secondary Indexes by Term

- Global index: covers data in all partitions
  - However, we can't just store that index on one node
  - Must also be partitioned to avoid hot spots

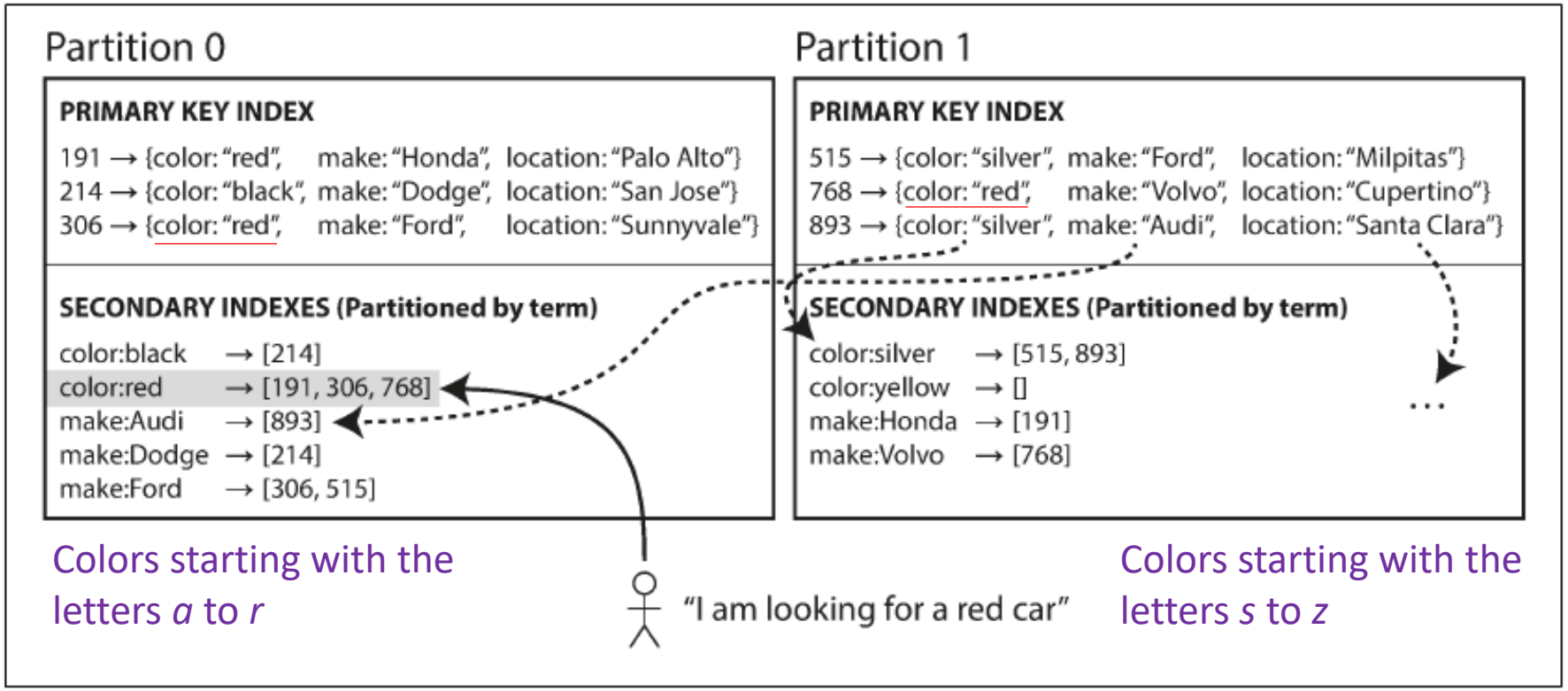


Figure 6-5. Partitioning secondary indexes by term.

Partitioning and Secondary Indexes:

Partitioning Secondary Indexes by Term

- The advantage of a global index over a document-partitioned index is that it can make reads more efficient:
  - Rather than doing scatter/gather over all partitions, a client only needs to make a request to the partition containing the term that it wants.

Partitioning and Secondary Indexes:

Partitioning Secondary Indexes by Term

- The downside of a global index is that writes are slower and more complicated:
  - A write to a single document may now affect multiple partitions of the index.
    - Every term in the document might be on a different partition, on a different node.
    - Updates to global secondary indexes are often asynchronous.
- Amazon DynamoDB, Riak's search feature.
- Oracle data warehouse lets you choose between local and global indexing.

Rebalancing Partitions

- Over time, things change in a database:
  - The query throughput increases
    - Add more CPUs to handle the load
  - The dataset size increases
    - Add more disks and RAM to store it
  - A machine fails
    - Other machines need to take over the failed machine's responsibilities
- The data and requests require to be moved from one node to another

Rebalancing Partitions

- The process of moving load from one node in the cluster to another is called *rebalancing*.
- Goals:
  - During rebalancing, reads and writes should be supported.
  - After rebalancing, load should be shared fairly between nodes.
  - Try to minimize the amount of data moved between nodes to make the rebalancing fast and to minimize the network and disk I/O load.

Rebalancing Partitions: Strategies for Rebalancing

- Fixed number of partitions
  - Create many more partitions than there are nodes.
  - Assign several partitions to each node.
  - If a new node is added, it can steal a few partitions from every existing nodes until partitions are fairly distributed once again.
  - If a node is removed – the same happens in reverse.
  - Figure 6-6
  - Is used in Riak, Elasticsearch, Couchbase, and Voldemort

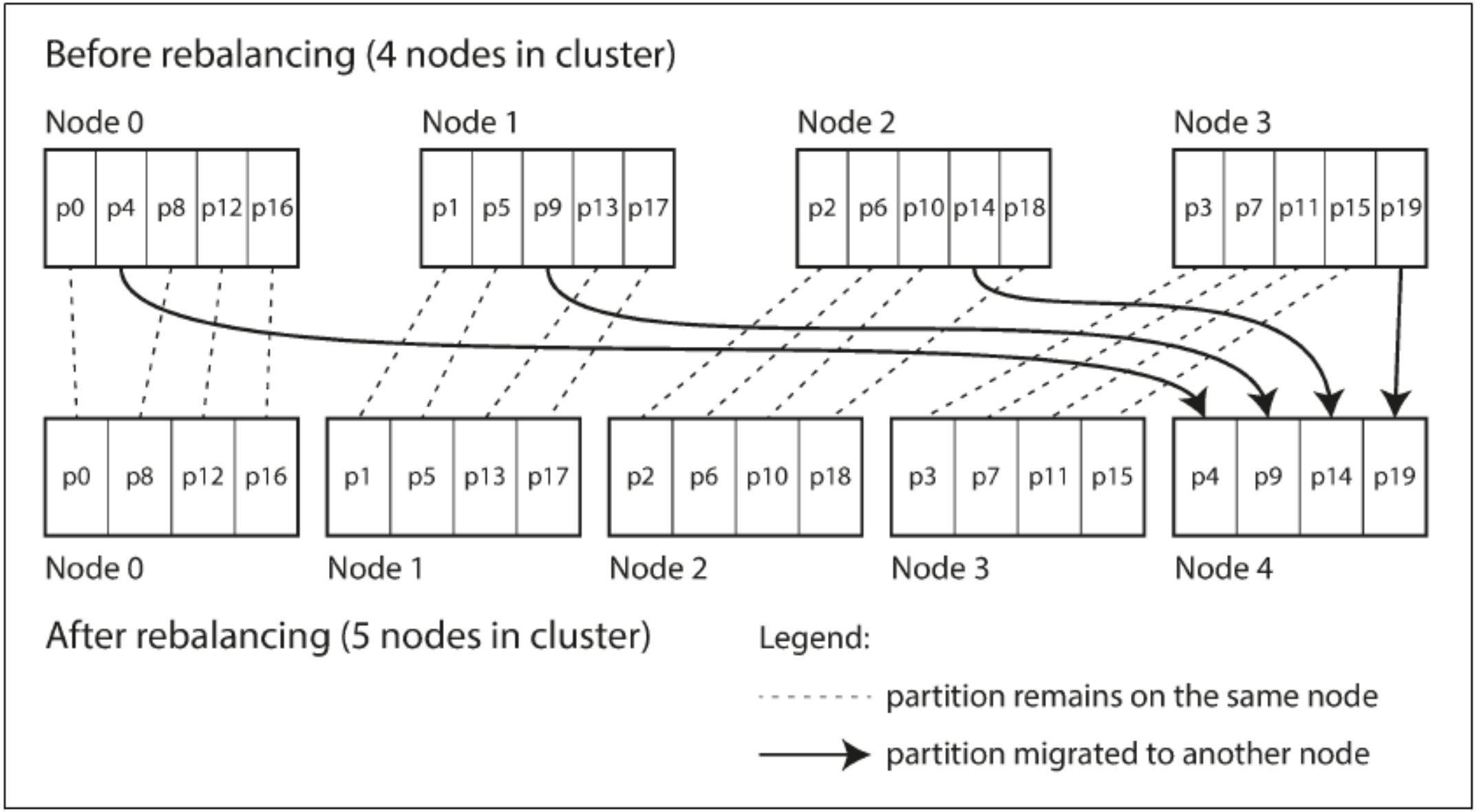


Figure 6-6. Adding a new node to a database cluster with multiple partitions per node.

Rebalancing Partitions: Strategies for Rebalancing

- Fixed number of partitions
  - The total number of partitions is usually fixed and not changed.
  - Choosing the right number of partitions is difficult if the total size of the dataset is highly variable.

Rebalancing Partitions: Strategies for Rebalancing

- Dynamic partitioning
  - For databases that use key range partitioning, a fixed number of partitions with fixed boundaries would be very inconvenient.
  - Dynamic partitioning splits a partition when it exceeds a certain size, and merges a partition with another when it drops below another size threshold.
  - The number of partitions grows with the size of the data.
    - You may want to initialize an empty database with one partition per node.

Rebalancing Partitions: Strategies for Rebalancing

- Dynamic partitioning
  - MongoDB since version 2.4
    - Supports both key-range and hash partitioning
    - Splits partitions dynamically in either case
  - With dynamic partitioning, the number of partitions is proportional to the size of the dataset.

Rebalancing Partitions: Strategies for Rebalancing

- Partitioning proportionally to nodes
  - The number of partitions are proportional to the number of nodes.
    - A fixed number of partitions per node.
  - The size of each partition grows proportionally to the dataset size while the number of nodes remains unchanged.
  - But when you increase the number of nodes, the partitions become smaller again.

Operations: Automatic or Manual Rebalancing

- Does the rebalancing happen automatically or manually? It can be
  - Fully automatic
  - Fully manual
  - In between (e.g., suggesting a partition reassignment, but requiring an admin to approve it)

Operations: Automatic or Manual Rebalancing

- Fully automatic is convenient
  - Less operational work for normal maintenance
- Risk of fully automatic:
  - Unpredictable
  - System performance could suffer if rebalancing is started at the wrong time
- It can be a good thing to have a human in the loop for rebalancing.
  - Slower than a fully automatic process (-)
  - Can help prevent operational surprises (+)

Request Routing

- When a client wants to make a request, how does it know which node to connect to?
- This is an instance of a more general problem called *service discovery*, which isn't limited to just databases.

Request Routing

- On a high level, there are a few different approaches to this problem:
  - Client sends to **any node**, and the node behind the scenes **forwards** the request to the proper partition when needed.
  - Client sends to an **intermediate routing tier**, a.nd that tier forwards the request
  - Clients **keep track of partitioning** and go directly to **correct node.**

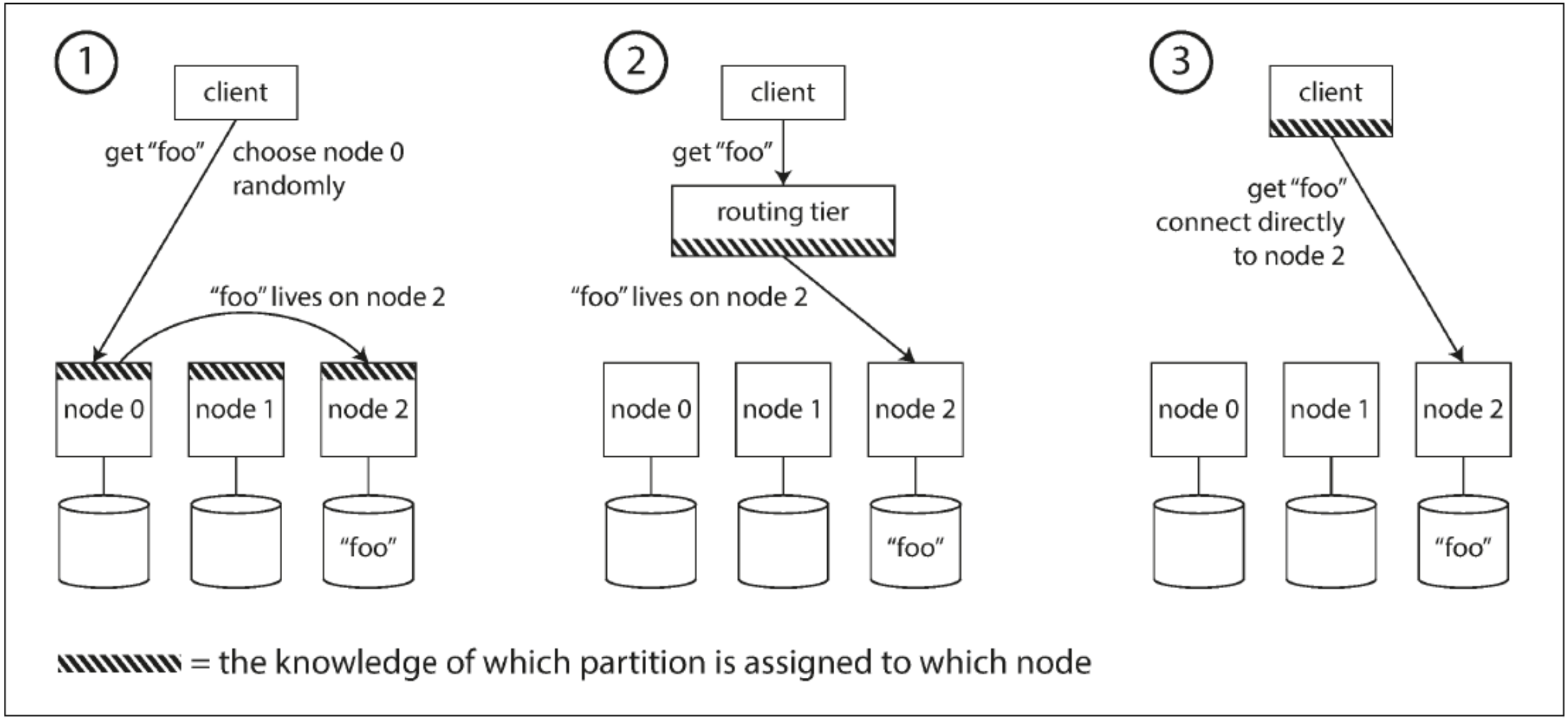
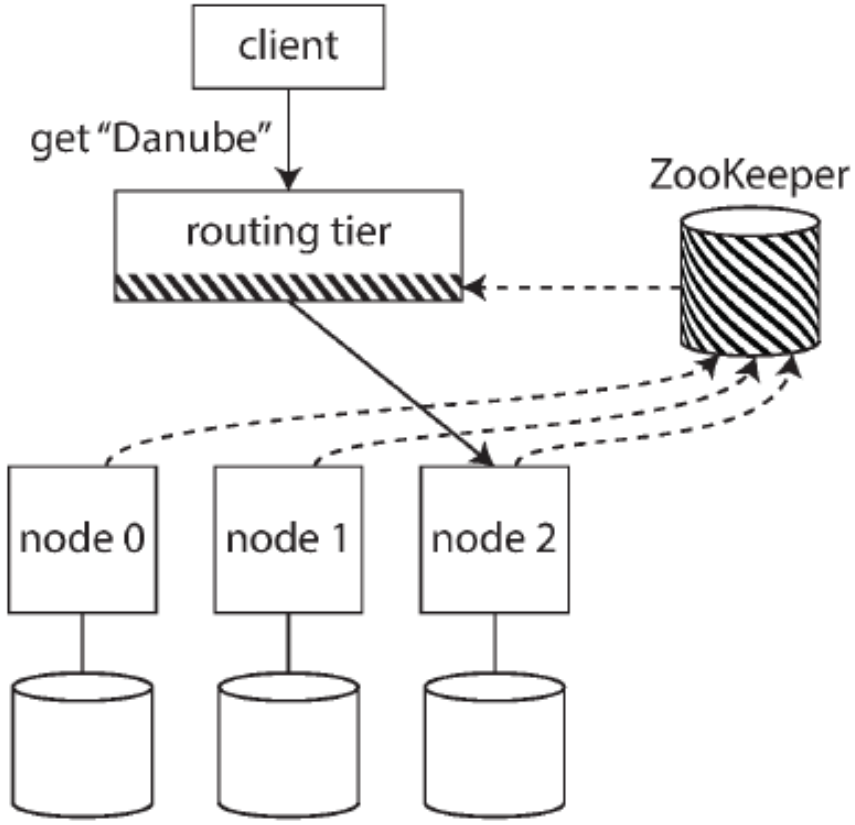


Figure 6-7. Three different ways of routing a request to the right node.

Request Routing

- In all cases, there will be nodes/tier/clients that need to keep track of partition changes.
- Many distributed data systems rely on separate coordination service called ZooKeeper to keep track of cluster/partition metadata.



Key range	Partition	Node	IP address
A-ak — Bayes	partition 0	node 0	10.20.30.100
Bayeu — Ceanothus	partition 1	node 1	10.20.30.101
Ceara — Deluc	partition 2	node 2	10.20.30.102
Delusion — Frenssen	partition 3	node 0	10.20.30.100
Freon — Holderlin	partition 4	node 1	10.20.30.101
Holderness — Krasnoje	partition 5	node 2	10.20.30.102
Krasnokamsk — Menadra	partition 6	node 0	10.20.30.100
Menage — Ottawa	partition 7	node 1	10.20.30.101
Otter — Rethimnon	partition 8	node 2	10.20.30.102
Reti — Solovets	partition 9	node 0	10.20.30.100
Solovyov — Truck	partition 10	node 1	10.20.30.101
Trudeau — Zywiec	partition 11	node 2	10.20.30.102

= the knowledge of which partition is assigned to which node

Figure 6-8. Using ZooKeeper to keep track of assignment of partitions to nodes.

References

- M. Kleppman. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, Inc., 2017.